챗봇 검증 결과 정리

2026-11-04 · 채영 · 브랜치 `scy` (아직 커밋 전) · 결과 파일 `eval/results/`

> 3rd 프로젝트의 평가(코드 변환 50문항)를 4th 코드로 다시 돌리고, **RAGAS**로 문서 질의응답(24문항)을 새로 평가했습니다.
> 평가에서 찾은 **검색 문제 3개를 고쳐서** 검색 적중률을 **0.62 → 0.95**로 올렸고,
> 지금 품질을 가장 많이 깎는 건 **분류기**라는 걸 확인해 개선안을 정리했습니다 (아직 안 고침, 팀 확인 필요).
> 시스템을 모르는 ChatGPT 가 만든 질문 두 세트로 다시 재도 **챗봇 > 일반 LLM** 은 같았고, 차이는 더 작게 나왔습니다 (6장).

코드 변환 공통 기준 통과	자체 평가	문서 검색 적중 (hit@6)	정답 사실 일치 (search)
63%	**91.8점**	**0.62→0.95**	**0.59→0.71**
일반 LLM 0%	A 등급 · 목표 19/21	검색 수정 후	일반 LLM 0.22

1. 무엇을 했나

작업	내용
① 코드 변환 평가 재실행	3rd 의 `eval/` (30 변환 + 6 다운그레이드 질문, 분류 프로브)을 4th 로 가져와 실행. 챗봇 vs 일반 LLM, 코드로 채점하는 공통 기준 8개 + 자체 평가 21항목.
② RAGAS 평가 추가	문서 질의응답 24문항을 새로 만들고(정답 문서 조각 ID 포함), RAGAS 5개 지표로 채점.
③ 검색 문제 3개 수정	RAGAS 결과에서 찾은 원인을 고침 (import 이름 검색, 0.3 가이드 감점 완화, 답변 프롬프트).
④ 분류기 개선안 작성	오분류 근거와 고치는 방법을 `docs/분류기_개선안.md` 에 정리. 코드는 아직 안 바꿈.

2. 평가 방법

2-1. 코드 변환 평가 (3rd 방식 그대로)

옛 LangChain 코드를 주고 1.x 로 바꾸게 한 뒤, 결과 코드를 **LLM 이 아니라 코드로** 채점합니다 (문법, 옛 import 없음, import 가 실제로 존재, 결과값 보존, 정답 import 포함 등 8개). 분류기가 질문을 맞는 경로로 보내는지(A1~A3)도 같이 봅니다.

2-2. RAGAS 문서 질의응답 평가 (새로 추가)

질문 24개: 마이그레이션 가이드 8 · 최신 문서 6 · import 위치 4 · 에러 메시지 3 · **문서에 없는 질문 3**(모른다고 답하는지 확인). 각 질문에 정답 답변과 정답 문서 조각 ID 를 붙였습니다. 채점관은 gpt-4o, 챗봇과 일반 LLM 은 같은 gpt-4o-mini.

지표	보는 것
hit@6	검색한 6개 조각 안에 정답 조각이 있나 (LLM 없이 ID 로 계산)
context recall / precision	정답에 필요한 내용을 찾아왔나 / 찾은 것 중 쓸모 있는 게 위에 있나
faithfulness	답이 찾아온 문서에 근거하나 (지어내지 않았나)
factual correctness	답이 정답과 사실이 일치하나 가장 중요하게 본 지표
answer relevancy	질문에 맞는 답인가

2-3. 두 가지 모드로 쟀습니다

search 모드 (부품 평가)
모든 질문을 search 경로로 보내 검색 + 근거 답변 품질만 봄.
"RAG 부품이 얼마나 좋은가"

e2e 모드 (실제 화면 기준)
그래프 전체를 돌려 분류기가 고른 경로의 답을 봄.
"사용자가 실제로 받는 답이 얼마나 좋은가"

두 모드의 차이 = 분류기가 질문을 잘못 보내서 생기는 손해. 그래서 검색 문제는 search 모드로, 분류 문제는 e2e 모드로 찾았습니다.

RAGAS 는 팀 `uv.lock` 과 패키지 버전이 충돌해서, 채점 스크립트만 따로 격리된 환경(`uv run --script`)에서 돌립니다. 팀 환경은 건드리지 않았습니다.

3. 코드 변환 평가 결과

항목	챗봇	일반 LLM
공통 기준 8개 모두 통과	63%	0%
옛 import 없음	97%	3%
import 가 실제로 존재	93%	3%
정답 import 포함	63%	0%

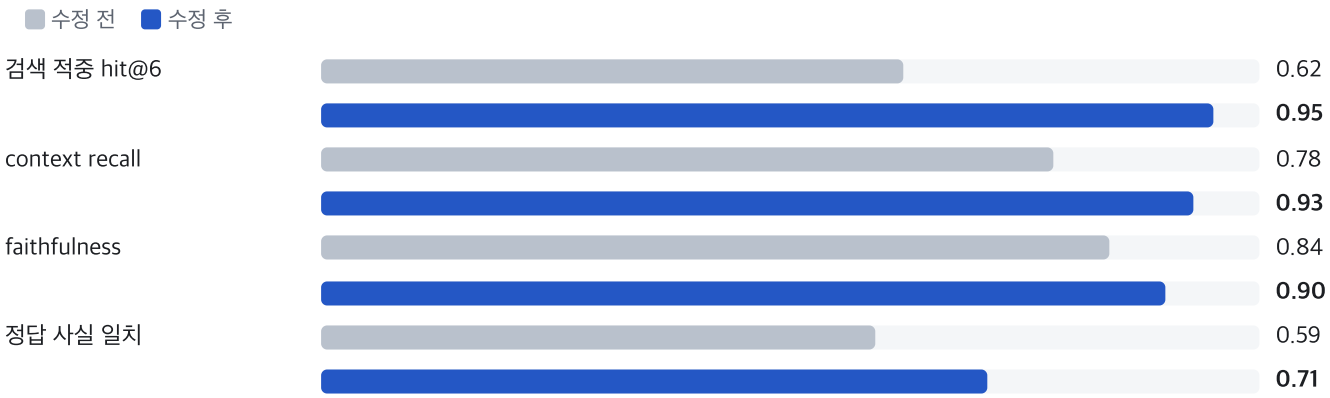
- 자체 평가 **91.8점 (A)**, 목표 21개 중 19개 달성.
- 못 채운 2개: **A1 코드 변환 요청을 migration 으로 분류 84%** (6개가 search 로 감), **E5 검증 통과 코드가 공통 채점도 통과 54%**.
- 실패 대부분(17개 중 15개)이 "정답 import 포함" — 에이전트 · 메모리 · RAG 유형이 약함.
- 평가 스크립트가 팀원 코드 변경(verifier 함수 인자)에 맞지 않아 `score.py` 를 한 줄 맞췄습니다.

4. RAGAS 로 찾은 검색 문제와 수정

4-1. 문제 3개와 원인

문제	원인	고친 것
이름만 묻는 import 질문에서 대응표를 못 찾음 "Chroma는 어디서 import해요?"	import 대응표(9,155개)는 `from langchain.x import Y` 처럼 import 문이 있어야만 찾게 되어 있음. 문장 속 이름만으로는 대응표를 안 봄.	문장 속 클래스 이름으로 대응표를 찾는 `lookup_names()` 추가 (search 경로에서만 켬)
마이그레이션 가이드를 못 찾음 "StuffDocumentsChain 대신 뭐 써요?"	코드 변환용으로 0.3 문서 점수에 ×0.8 감점을 줌. 문서 질문에서는 정답인 0.3 마이그레이션 가이드가 1위인데도 6위 밖으로 밀려남.	search 경로만 0.3 문서 감점을 ×0.9 로 완화 (코드 변환 경로는 그대로)
너무 조심스러운 답	찾은 문서로 답할 수 있는데도 "확인되지 않습니다"로 시작. 프롬프트가 "충분하지 않으면 모른다고" 만 강조.	search 프롬프트 수정: 일부라도 답할 수 있으면 결론부터, import 대응표는 확정 근거로 쓰라고 명시

4-2. 수정 전후 (search 모드, 답이 있는 21문항)

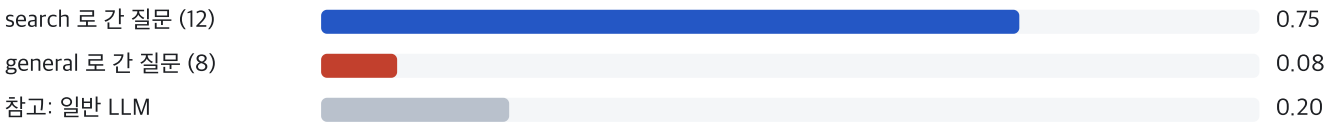


유형별 hit@6	수정 전	수정 후	정답 사실 일치 (후)	일반 LLM
import 위치 (4)	0.25	**1.00**	0.52	0.05
마이그레이션 가이드 (8)	0.50	**1.00**	0.65	0.25
최신 문서 (6)	0.83	0.83	0.81	0.25
에러 메시지 (3)	1.00	1.00	0.89	0.30
문서에 없는 질문 (3) — 모른다고 답함	3/3	3/3		0/3

질문 하나 단위 점수는 채점관 편차가 ±0.3 정도라, 평균으로 비교했습니다. 코드 변환 경로(migration)는 기본값이 그대로라 영향 없음.

5. 남은 핵심 문제: 분류기

search 모드에서 0.71 이 나오는데, **실제 화면 기준(e2e)은 0.47** 입니다. 차이는 분류기가 만듭니다.



general 로 간 8개는 모두 **마이그레이션 질문**인데 general 로 빠져서, 검색 없이 모델 기억으로 답했습니다. 같은 질문을 search 로 보내면 **0.69** 가 나옵니다.

id	질문	지금 (general)	search 였다면
r01	LLMChain은 이제 안 쓴다던데, 대신 어떻게 만들어요?	0.00	0.14
r02	RetrievalQA 체인을 LCEL로 바꾸면 뭐가 좋아지고 구조는?	0.25	0.46
r04	AgentExecutor를 LangGraph로 옮기면 뭐가 그 역할을 해요?	0.00	0.80
r05	AgentExecutor의 max_iterations=3 을 LangGraph에서는?	0.00	0.83
r08	LangGraph로 메모리를 관리하면 어떤 장점이 있어요?	0.14	0.75
r09	langchain-classic 패키지는 왜 생겼고 언제 설치해요?	0.27	0.83
r11	LangChain 1.0은 파이썬 몇 버전부터 돼요?	0.00	0.80
r14	pre_model_hook 으로 요약하던 건 create_agent에서 어떻게?	0.00	0.89

그 밖에 코드 없는 질문 1개(r03)가 migration 으로 가서 질문 문장을 코드로 "변환"했고, 코드 변환 평가에서는 옛 코드 + "에러가 나요" 요청 6개가 search 로 가서 **검증 루프를 안 거쳤습니다** (A1 84%).

Q. search 로 보내도 우리 DB에는 마이그레이션 데이터밖에 없는데 괜찮나?

괜찮습니다. 위 8개는 모두 마이그레이션 질문이고, 정답 조각이 **이미 지금 데이터에 있습니다** (search 로 보냈을 때 8개 중 7개가 정답 조각을 찾음). 고치는 건 데이터가 아니라 **어떤 질문을 RAG 로 보낼지**입니다. 참고로 지금 데이터 구성은 아래와 같습니다.

출처	조각 수	내용
import 대응표	9,155	옛 경로 → 1.x 경로
1.x 최신 문서	2,689	마이그레이션 · 릴리스 · 버전 문서 약 100 + integrations 1,996 + 사용법 546 등
0.3 마이그레이션 가이드	59	"Migrating from LLMChain" 등
메서드 대응표	42	`.run()` → `.invoke()` 등

5-1. 원인 (classifier_prompt.py · classifier_node.py)

- **규칙 4 "어떻게 쓰는가 → general"**: 마이그레이션 질문도 대부분 "LLMChain 대신 어떻게?" 형태라 general 로 빠짐.
- **규칙 6 "애매하면 general"**: 질문에 1.x 에서 밀려난 옛 API 이름(LLMChain, AgentExecutor 등)이 나오는 강한 신호를 안 씀.
- search 범위 설명이 좁음: 지원 Python 버전, langchain-classic 같은 업그레이드 요건이 예시에 없음.
- "코드가 있어야 migration", "코드 + 에러 → migration" 같은 코드로 확인할 수 있는 조건을 LLM 에만 맡김.
- 분류기 `temperature=0.7`, 출력값 검사 없음.

5-2. 개선안 (팀 결정: 마이그레이션 질문만 search, 나머지는 모두 general — 범위는 그대로)

```
LLM 에 묻기 전에 코드로 먼저 정하기
1. 옛 import 문이 있고 + 변환 의도(바꿔·고쳐·에러·설치·업그레이드 …)  → migration 확정
2. 코드 블록도 import 문도 없음                                    → migration 금지
3. 질문 문장에 옛 API 이름(대응표의 classic_fallback 499개)이 있음     → general 금지 (search)
4. 나머지                                                          → LLM 분류

프롬프트: 규칙 4 에 "옛 방식 대신 / 옮기면" 은 마이그레이션 질문이라는 예외 + search 예시 추가
안정성: temperature 0, 출력 정리 후 세 값 밖이면 general
```

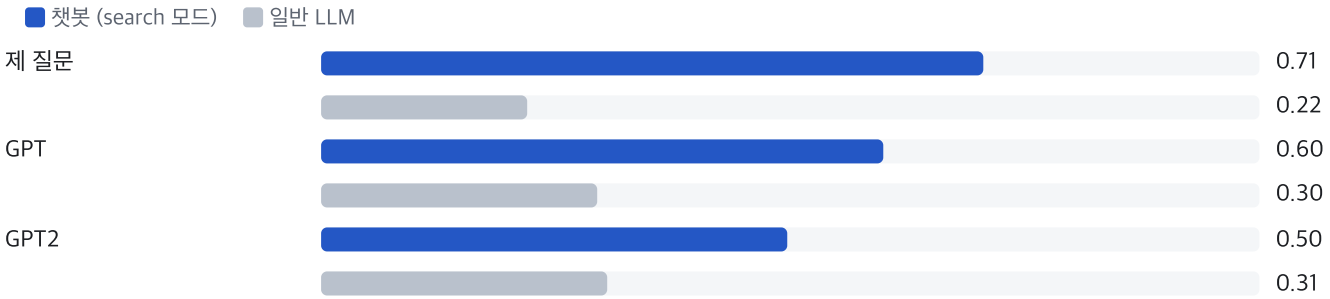
- 코드 규칙만으로 r01 · r02 · r04 · r05 · r14 · r03 과 A1 오분류 6개가 바로잡힘 (코드로 확인함). r08 · r09 · r11 은 프롬프트 수정에 달림.
- 회귀 확인: general 이어야 하는 프로브 질문(인사, "LCEL이 뭐야?", 다른 라이브러리, 지시문 주입 등)은 규칙에 하나도 안 걸림.
- 새 데이터 없음. 검색기가 쓰는 `lookup()` · `lookup_names()` 를 그대로 씀.

기대 효과 (지금 데이터로 추정)	지금	고친 뒤
e2e 정답 사실 일치	0.47	≈ 0.71
A1 코드 변환 요청 → migration	84%	100% 목표

6. 다른 출제자의 질문으로 다시 검증

4~5장의 24문항은 제가(검색을 고친 사람이) 만들어서 점수가 유리할 수 있습니다. 그래서 시스템을 모르는 ChatGPT에게 질문을 두 번 만들게 하고, **내용을 고치지 않고** 같은 방식으로 채점했습니다. GPT2 는 GPT 세트의 문제(출처가 한 페이지에 몰림, 범위 밖 질문 섞임)를 막는 규칙을 넣은 프롬프트로 만들었습니다.

정답 사실 일치 (답이 있는 질문)	제 질문 (21)	GPT (27)	GPT2 (26)
챗봇 · search 모드	0.71	0.60	**0.50**
챗봇 · e2e (실제 화면)	0.47	0.46	**0.48**
일반 LLM	0.22	0.30	0.31
챗봇 – 일반 LLM (search)	+0.49	+0.31	+0.19
필요한 문서 내용을 찾아온 비율 (context recall)	0.93	0.80	0.69
마이그레이션 질문이 search 밖으로 간 수	9	8	1
문서에 없는 질문에 '모른다' (search)	3/3	1/3	4/4



6-1. 알게 된 것

- **챗봇 > 일반 LLM 은 세 세트 모두 같습니다.** 다만 차이는 출제자가 시스템에서 멀어질수록 줄어듭니다 (+0.49 → +0.31 → +0.19). 제 질문 점수는 부풀려진 것이 맞고, 실제 성능은 GPT2 쪽 숫자에 가깝다고 보는 것이 안전합니다.
- **분류 오류는 질문 말투에 달려 있습니다.** GPT2 질문은 거의 모두 "1.x로 올리니까", "업그레이드하고" 처럼 버전 단어가 있어 분류기가 잘 맞혔고(오분류 1개), 제 질문 · GPT 질문은 "LLMChain 대신 어떻게?" 처럼 버전 단어 없이 물어 8~9개가 general 로 빠졌습니다. 5장 개선안(옛 API 이름 → general 금지, "대신 · 옮기면" 예외)이 노리는 것이 바로 이 말투라, 개선안은 여전히 필요합니다.
- **데이터에 없는 마이그레이션 문서가 있습니다.** `chunks.jsonl` 을 직접 찾아보니 **LangGraph v1 마이그레이션 가이드** (`ValidationNode` , `AgentStatePydantic` 등)가 0개입니다 (GPT2 h06~h09). LangGraph 메모리 문서(h27~h30)도 없지만 이건 범위 밖이라 괜찮습니다.
- 데이터에 내용이 있는데 검색이 놓친 질문도 있습니다 (GPT2 h03 TypedDict, h12 `content_blocks` , GPT g03 · g04). 문항 몇 개에 맞춰 고치면 다시 '본 문제'가 되므로 보류합니다.
- '모른다' 판정: GPT 세트 g28 은 답이 "없습니다"로 시작했는데 판정 정규식이 놓쳤습니다 (채점 도구 문제).

모든 모델이 OpenAI 계열입니다 (챗봇 · 일반 LLM gpt-4o-mini, 채점 gpt-4o). 같은 계열 채점관 편향은 챗봇과 일반 LLM 둘 다에 걸리므로 '차이'는 비교적 안전하지만, 다른 회사 모델(예: Claude)로 한 번 더 채점하면 더 확실해집니다. GPT 가 쓴 정답(reference)은 아직 사람이 공식 문서와 대조하지 않았습니다.

7. 팀에서 정할 것

#	안건	상태 / 추천
1	분류기를 개선안대로 고칠지	확인 받으면 바로 수정 후 재평가
2	"코드 + 에러가 나요" 를 migration 으로 볼지 (지금 프롬프트는 search, 평가 기준은 migration)	추천: migration — 코드를 붙인 사용자는 돌아가는 코드를 원함
3	general 이 문서에 없는 옵션을 지어냄 (r23 LangSmith 요금, r24 없는 ChatOpenAI 옵션)	general 프롬프트에 "모르면 모른다" 추가 검토
4	LangGraph v1 마이그레이션 가이드를 데이터에 추가할지	추천: 추가 — 범위 안 공식 문서. 효과는 새 질문으로 재기
5	GPT 질문 정답(reference)을 공식 문서와 대조	사람이 확인 필요 (근거 문장 `evidence_quote` 첨부됨)
—	LCEL 개념 등 마이그레이션 외 질문	결정됨: general

평가 도구 문제: `run_eval.py` 가 질문마다 thread_id 를 안 넘겨 50개 질문이 한 대화로 이어지던 것 → **고침** (질문마다 새 thread). 코드 변환 평가 결과(3장)는 고치기 전에 잰 것이라 다시 재야 합니다.

8. 바뀐 파일과 실행 방법

파일	내용
`eval/` (3rd 에서 가져옴)	코드 변환 평가. import 를 `langration.src` 로 맞추고 `score.py` 한 줄 수정
`eval/ragas_questions.json`	RAGAS 질문 24개 (정답 · 정답 조각 ID)
`eval/run_ragas.py` · `score_ragas.py`	답 모으기 (search / e2e) · RAGAS 채점 (격리 환경). `--set` 으로 질문 세트 선택
`eval/ragas_questions_gpt.json` · `_gpt2.json`	ChatGPT 가 만든 질문 (수정 없이 저장). 결과는 `eval/results/gpt/` · `gpt2/`
`eval/run_eval.py`	질문마다 thread_id 따로
`langration/src/rag/retrieval/import_lookup.py`	`lookup_names()` 추가
`langration/src/rag/retrieval/hybrid.py`	`retrieve()` 에 `name_lookup`, `era_weight` 옵션 (기본값은 그대로)
`langration/src/graph/nodes/search_node.py`	search 경로에서만 위 옵션 켬
`langration/src/prompt/search_prompt.py`	답변 원칙 수정
`docs/분류기_개선안.md`	분류기 근거 · 개선안 상세
`.gitignore`	`data/vectordb/` 추가

```
uv run python eval/run_eval.py                    # 코드 변환 평가
uv run python eval/score.py
uv run python eval/run_ragas.py                   # RAGAS search 모드 (--mode e2e 로 실제 화면 기준)
uv run --script eval/score_ragas.py               # 채점 (OPENAI_API_KEY 필요, .env 사용)
```